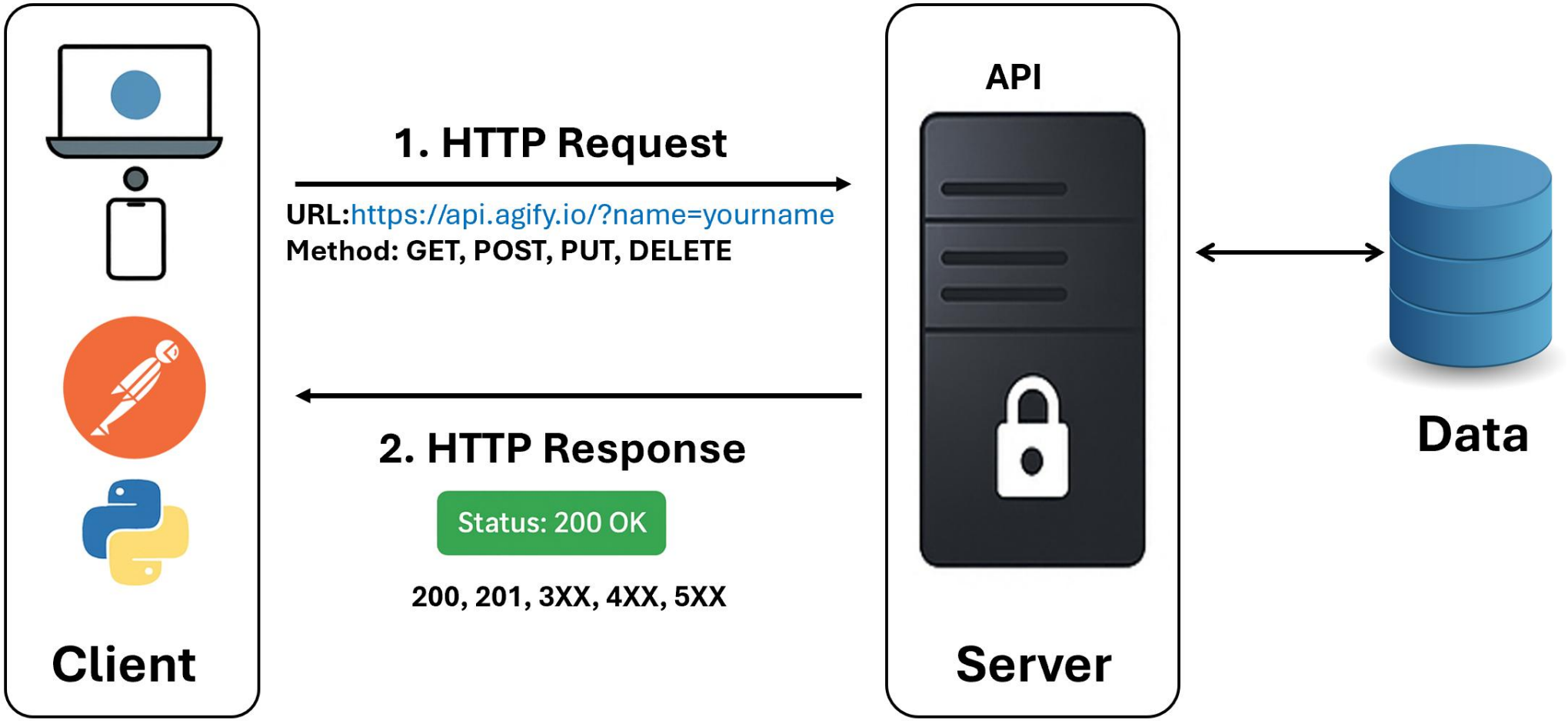
CSIS 1230-01 PROGRAMMING FOR EVERYONE II

Introduction to Web APIs

OUTLINE

- What clients and servers are, and how they talk using HTTP
- What a URL and query parameters work
- Core HTTP methods (GET, POST, PUT, DELETE)
- HTTP status codes (2XX, 4XX, 5XX)
- Basics of authentication (API keys, Bearer tokens)
- Handson with browser and Postman

CLIENT – SERVER COMMUNICATION VIA HTTP



CLIENT – SERVER (RESTAURANT)

CLIENT	API	SERVER
You have a need (food).	List of rules for what to order	Receives your order
You look at the menu to see what you can order	Tells you what dishes you can order	Prepares the food exacts as described on the menu
Send your order to the server.	How to ask for each by name	
Wait for your food	What you will get back	Server to You (Client)

CLIENT – SERVER (WEB TERMS)

- **Client**: Your web browser, Python program, Postman app, etc.
  Basically, anything that requests data.

- **Server**: A powerful computer running somewhere in the world that responds with data.

- **API:** (Application Programming Interface): The "menu" or "contract" that defines how a client and server can talk to each other.

CLIENT – SERVER (DEMO)

- **Client**: Your web browser makes a request to https://www.northpark.edu/

  (REQUEST)

- **Server:** North Park's servers send back HTML, CSS and images

  (RESPONSE)

- Your browser assembles it all into what you see.

HTTP

- Hypertext Transfer Protocol.
- The language the client uses to communicate with the server.
- The set of rules (a protocol) that allows your computer (or phone, browser, app, etc.) to talk to servers on the internet, fetch data like web pages or APIs, send data, and display results.

ANATOMY OF A URL

Uniform Resource Locator:

Full address of a specific resource on the internet.

Example; https://jsonplaceholder.typicode.com/posts?userId=1

- **https** → protocol (secure HTTP)
- **jsonplaceholder.typicode.com** → server (domain)
- **/posts** → path (resource) on the server
- **?userId=1** → query parameter (`userId` equals 1) – returns record(s) where userId is 1

HTTP METHODS

Method	Function
GET	Retrieve this information (e.g., “GET the weather for Chicago”)
POST	Here is some information, please create a new record (e.g., “POST a new tweet”)
PUT	Update this existing information. (e.g., “UPDATE my user profile”)
DELETE	Please delete this information. (e.g., “DELETE this post”)

STATUS CODES

- 2XX Success: 200 OK (`GET/PUT`), 201 Created (`POST`), 204 No Content (`DELETE`)
- 4XX Client Errors: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
- 5XX Server Errors: 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Why they matter?

Your code should check the status code and handle errors nicely.

STATUS CODES

Code	Name	Meaning	Common Causes
200	**OK**	Success!	The request worked perfectly (e.g., your AAPL ticker data was returned).
400	**Bad Request**	Client Error	The server didn't understand the request (often due to a syntax error in your JSON).
401	**Unauthorized**	Client Error	You aren't authenticated. In your case, it's likely that **GitHub PAT** being invalid or expired.
403	**Forbidden**	Client Error	You are authenticated, but you don't have permission to see this specific resource.
404	**Not Found**	Client Error	The specific URL or endpoint you're hitting doesn't exist.
500	**Internal Server Error**	Server Error	A generic "something went wrong" on the server's end.
502	**Bad Gateway**	Server Error	One server received an invalid response from another server it was trying to talk to.

AUTH

- **API Key:** a secret string you send (often in a header)
- **Basic Auth:** username, password (only over `HTTPS`)
- **Bearer Token / OAuth2/ JWT:** You send a token in the `Authorization` header

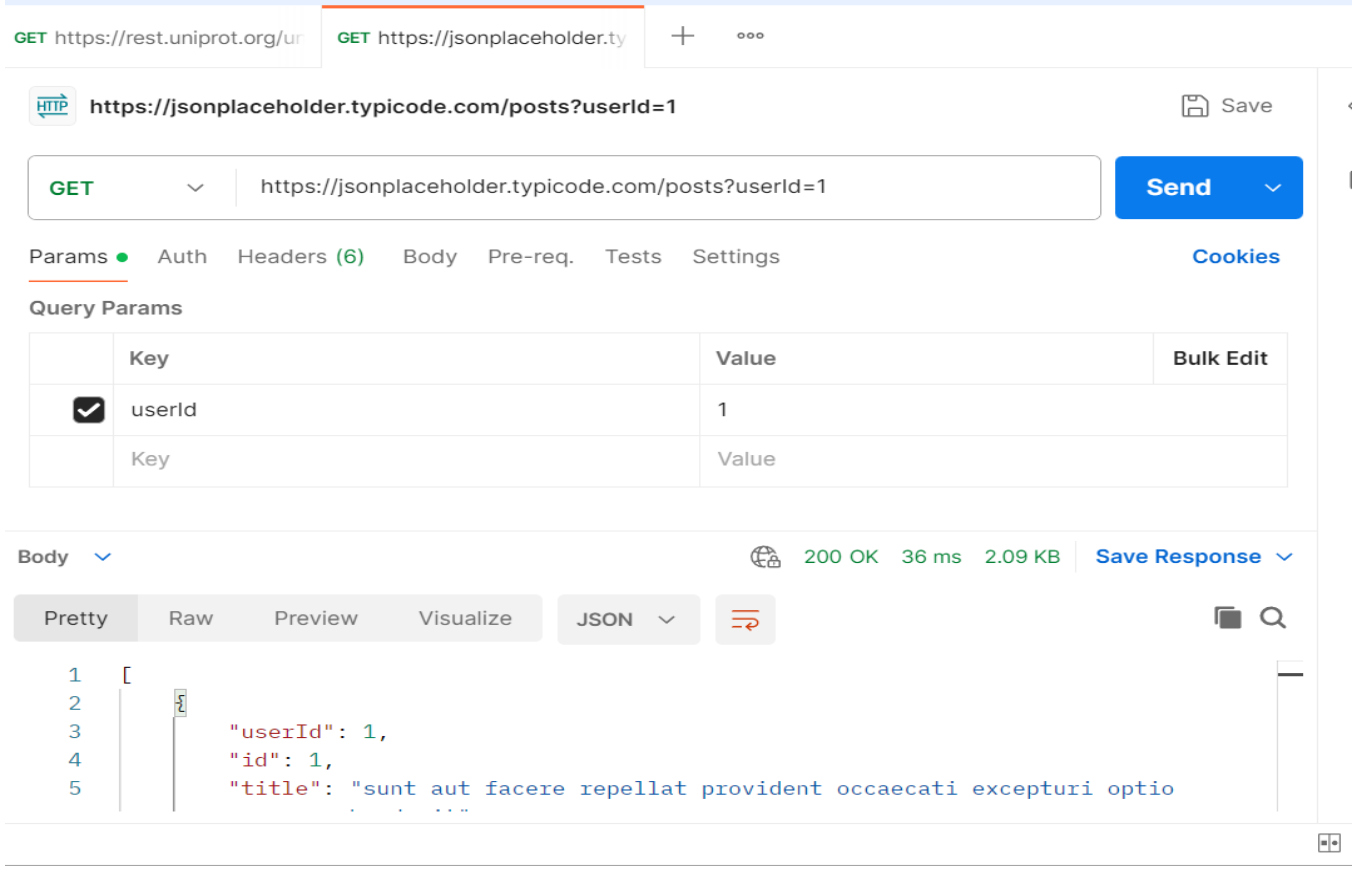
CLASS EXERCISE

- Download postman or use your web browser
- Make HTTP Request for data available at the following URLs or End points

- https://jsonplaceholder.typicode.com/todos?userId=2
- https://jsonplaceholder.typicode.com/posts?userId=1
- https://reqres.in/api/users?page=1

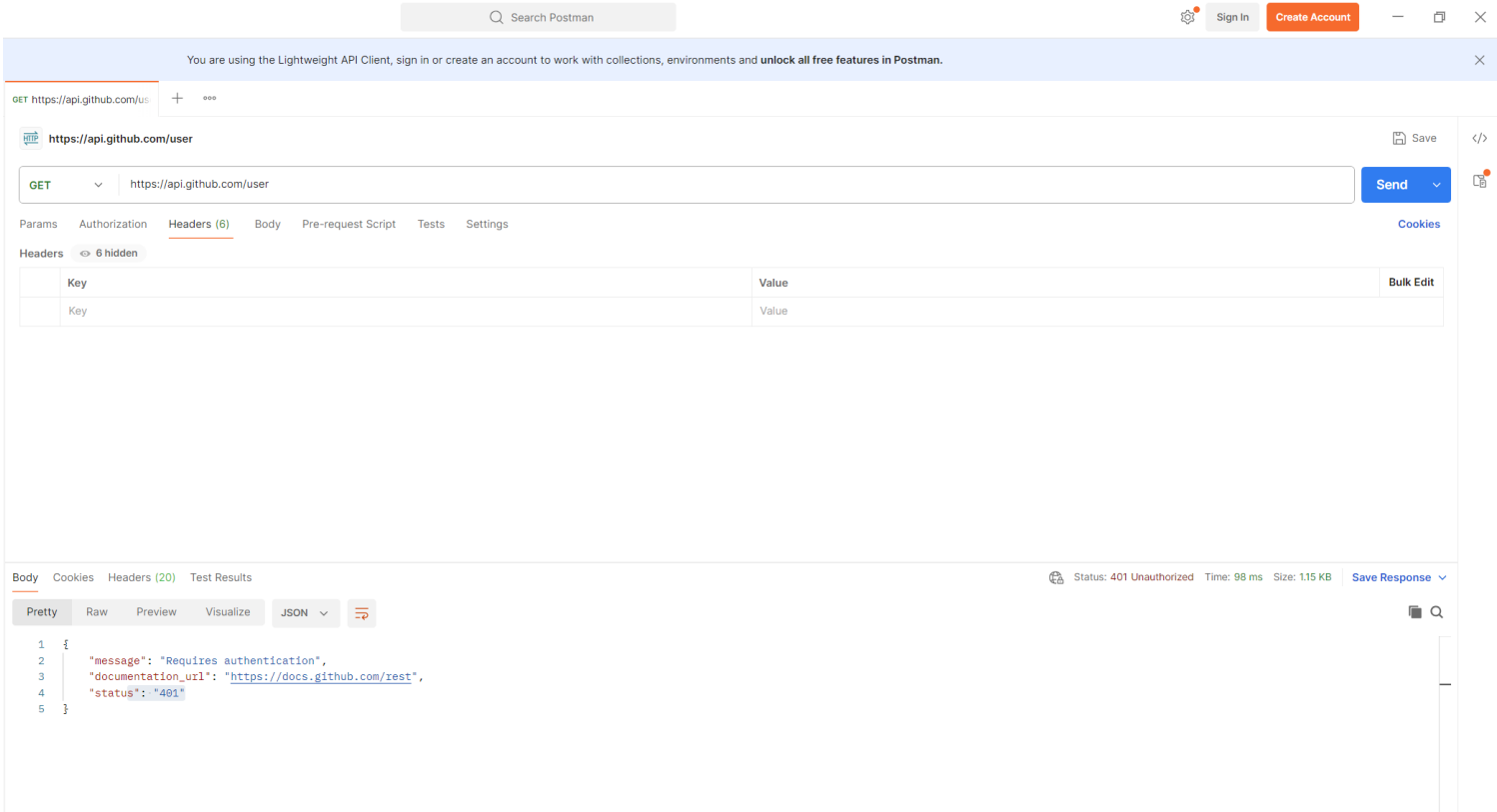
FREE API

- https://jsonplaceholder.typicode.com/posts?userId=1



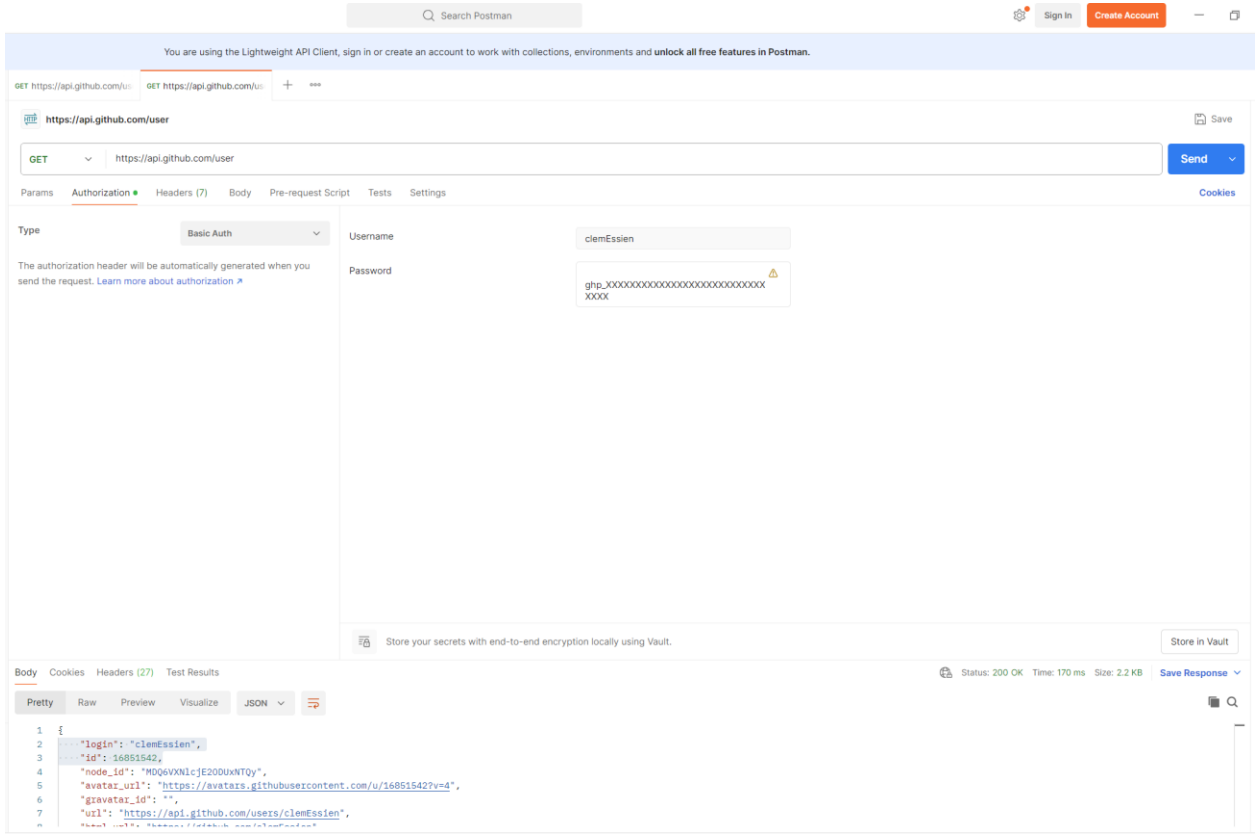
AUTH REQUIRED

- https://api.github.com/user



AUTH REQUIRED (FIXED)

- https://api.github.com/user



GOOD HABITS

- Use the right HTTP method for the job (GET for read, POST for create, etc.).
- Always check the status code and handle non-2xx cases.
- Use timeouts to avoid hanging requests.
- Keep API keys/tokens out of public code; prefer environment variables.
- Read the API docs: required parameters, auth, rate limits, and example responses.

QUIZ

- In **HTTP**, which part tells the server what action you want (read/create/update/delete)?
- What is the difference between **200 OK** and **201** Created?
- **True/False**: GET should change data on the server.
- Which header often carries tokens for APIs (e.g., OAuth2/JWT)?
- What is a query parameter? Give an example using ?key=value.

SUMMARY

- **Client** and **Server** talk via **HTTP** (request → response).
- **Methods** describe the action; status codes describe the result.
- **URLs + query parameters** control what you ask for.
- You used a browser, Postman, and Python to call real APIs (**JSONPlaceholder**, **Open-Meteo**).

NEXT

- Use Python's **`requests`** library to make:
  - GET
  - POST
  - PUT
  - DELETE

QUESTIONS